Installation locale

Ce guide fait tourner Candex sur votre machine : l'**API** avec une base PostgreSQL locale, l'**application mobile** et le **tableau de bord d'administration**. Si ce n'est pas déjà fait, parcourez d'abord Travailler sur Candex pour la vue d'ensemble.

La base locale est construite **de zéro** par les migrations Flyway de l'API (qui contiennent des seeds) : vous n'avez donc **aucun** besoin d'accès à un serveur.

Ce que vous aurez en route

Service	URL
API REST	`http://localhost:8080` (Swagger sur `/swagger-ui.html`)
Application mobile	`http://localhost:8100`
Tableau de bord admin	`http://localhost:8101`
PostgreSQL	`localhost:5432`

Prérequis

- **Git**
- **Node.js 20+** et **npm 10+** — pour l'app et le dashboard
- **JDK 21** — pour l'API (un wrapper Maven, `mvnw`, est inclus, donc une installation Maven séparée est optionnelle)
- **Docker** et Docker Compose — pour la stack locale Postgres + API
- La config d'un **projet Firebase** — pour l'authentification de l'app (voir la section app)
- **Android Studio** — optionnel, uniquement pour les builds natifs Android

Récupérer le code

Clonez les dépôts dans un même dossier :

```
git clone https://github.com/RusterX16/candex-api
git clone https://github.com/RusterX16/candex-app
git clone https://github.com/RusterX16/candex-dashboard
```

1 — API et base de données (`candex-api`)

Créez le fichier d'environnement à partir du modèle et renseignez les valeurs :

```
cd candex-api
cp .env.example .env     # puis éditez .env
```

Le chemin recommandé est le script d'aide, qui monte une stack conteneurisée jetable (Postgres 17 + API) depuis `compose.local.yaml` :

```
./scripts/candex.sh up        # démarre Postgres + API en local
```

Au démarrage, l'API exécute ses **migrations Flyway**, qui créent le schéma et insèrent les données de base (marques, versions d'app, etc.) — une base fonctionnelle sans aucun accès serveur.

Autres commandes utiles :

```
./scripts/candex.sh down      # arrête la stack (données conservées)
./scripts/candex.sh reset     # efface le volume de la base locale
./scripts/candex.sh psql      # ouvre psql sur la base locale
./scripts/candex.sh logs      # suit les logs de l'API
```

L'API est alors sur `http://localhost:8080`, avec Swagger UI sur `/swagger-ui.html`.

> **Sans Docker**
>
> Si vous préférez votre propre PostgreSQL, définissez `CANDEX_DB_URL`, `CANDEX_DB_USERNAME` et `CANDEX_DB_PASSWORD`, puis démarrez l'API avec `./run-dev.sh` (ou `./mvnw spring-boot:run -Dspring-boot.run.profiles=dev`).

2 — Application mobile (`candex-app`)

Installez les dépendances :

```
cd candex-app
npm install
```

L'app génère son environnement Angular à partir de fichiers env (`scripts/generate-env.js`). Créez-les à partir du modèle :

- `.env.dev` et `.env.prod` — un projet Firebase chacun, plus `API_URL` (copiez `.env.example`)
- `.env.local` — juste `API_URL=http://localhost:8080`, qui écrase l'URL de l'API pour les runs locaux

Config Firebase. L'app s'authentifie via Firebase : `.env.dev` / `.env.prod` ont besoin des sept valeurs `FIREBASE_*`, et le build Android a besoin de `android/app/google-services.json`. Récupérez-les depuis votre projet Firebase (Paramètres du projet → votre app web) ou auprès du owner du projet. Aucune valeur Firebase n'est incluse dans cette documentation.

Lancez contre votre API locale :

```
npm run start:local     # http://localhost:8100 (Firebase dev + API localhost)
```

`start:local` s'appaire avec le profil **dev** de l'API — gardez les deux en `dev`. Pour un build natif Android (nécessite Android Studio) : `npm run android:dev`.

3 — Tableau de bord admin (`candex-dashboard`)

```
cd candex-dashboard
npm install
cp .env.example .env     # renseignez API_URL_LOCAL et les identifiants admin
npm start                # http://localhost:8101
```

Tout assembler

Pour un setup local cohérent, gardez ensemble le mode **local** de l'app et le profil **dev** de l'API. Ordre typique : démarrez l'API (`candex.sh up`), puis le dashboard (`npm start`) et l'app (`npm run start:local`).

Dépannage

- **Port déjà utilisé** — arrêtez le processus qui occupe `8080`, `8100`, `8101` ou `5432`, ou changez le port.
- **L'API n'atteint pas la base** — vérifiez que Docker tourne et que `candex.sh up` rapporte un `db` sain ; contrôlez les valeurs `DEV_DB_*` dans `.env`.
- **Le build de l'app échoue avec « Missing env vars »** — `generate-env.js` a besoin de `API_URL` et des sept valeurs `FIREBASE_*` dans le fichier `.env.*` chargé.
- **Base vide** — les migrations s'exécutent au démarrage de l'API ; vérifiez `candex.sh logs` pour confirmer que Flyway les a appliquées.

Où aller ensuite

- Internes de l’API → Back-end *(à venir)*
- Environnements et releases → Déploiement *(à venir)*